DATA + FOLLOW THIS TOPIC

Square off: Machine learning libraries

Top five characteristics to consider when deciding which library to use.

By Mayukh Bhaowal. January 16, 2018

Main reading room at the Library of Congress
(source: Library of Congress on Flickr)

Check out the session, "The Journey of Machine Learning Platform Adoption in Enterprise," at Strata London, May 21-24, 2018. Hurry—best price ends February 23.

Choosing a machine learning (ML) library to solve predictive use cases is easier said than done.

There are many to choose from, and each have their own niche and benefits that are good for specific use cases. Even for someone with decent experience in ML and data science, it

can be an ordeal to vet all the varied solutions. Where do you start? At Salesforce Einstein, we have to constantly research the market to stay on top of it. Here are some observations on the top five characteristics of ML libraries that developers should consider when deciding what library to use:

1. Programming paradigm

Most ML libraries fall into two tribes on a high-level design pattern: the symbolic tribe and the imperative tribe for mathematical computation.

STRATA DATA CONFERENCE

Strata Data Conference in London, May 21-24, 2018

Best price ends February 23.

In symbolic programs, you define a complex mathematical computation functionally without actually executing it. It generally takes the form of a computational graph. You

lay out all the pieces and connect them in an abstract fashion before materializing it with real values as inputs. The biggest advantages of this pattern are composability and abstraction, thus allowing the developers to focus on higher level problems. Efficiency is another big advantage, as it is relatively easy to parallelize such functions.

Apache Spark's ML library, Spark MLlib, and any library built on Spark, such as Microsoft's MMLSpark and Intel's BigDL, follow this paradigm. Directed acyclic graph (DAG) is their representation of the computational graphs. Other examples of symbolic program ML libraries are CNTK, with static computational graphs; Caffe2, with Net (which is a graph of operators); H2O.ai; and Keras.

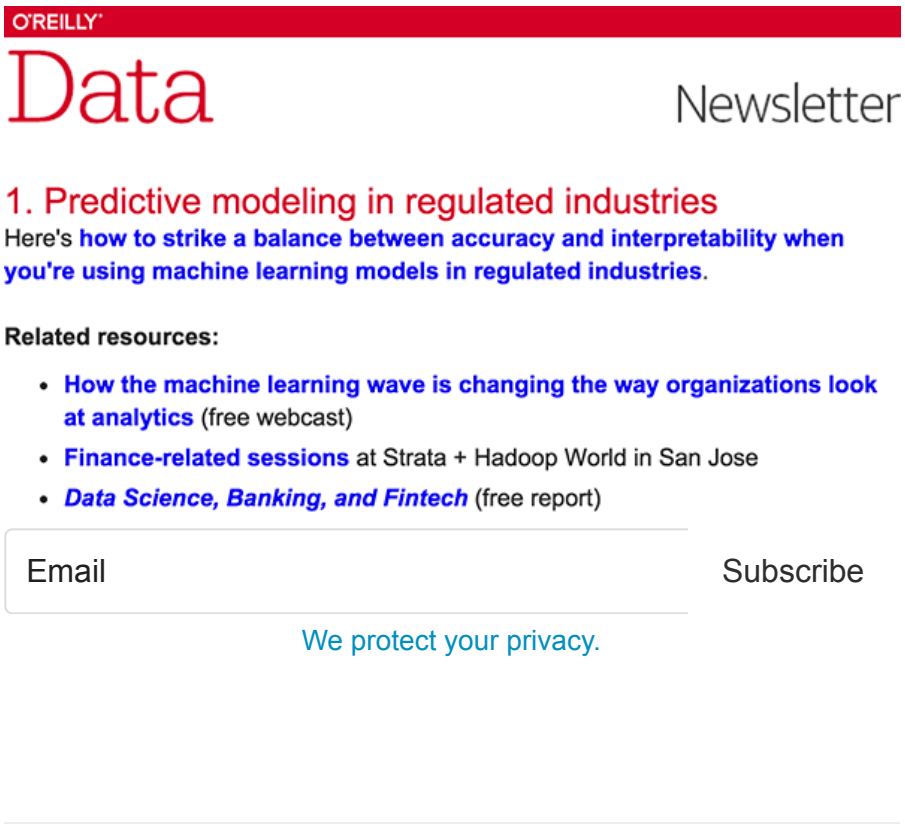
In imperative programs, everything is execution-first. You write a line of code, and when the compiler reads that line and executes it, it actually runs the numerical computation and moves to the next line of code. This style makes prototyping much easier, as it tends to be more flexible and much easier to debug and troubleshoot. Scikit-learn is a popular Python library that falls into this category. Other libraries such as auto sklearn and TPOT are layers of abstraction on top of scikit-learn, which also follow this paradigm. PyTorch is yet another popular choice that supports dynamic computational graphs, thereby making the process imperative.

Clearly, there are tradeoffs with either approach, and the right one depends on the use case. Imperative programming is great for research, as it naturally supports faster prototyping—allowing for repetitive iterations, failed attempts, and a quick feedback loop—whereas symbolic programming is better catered toward production applications.

There are some libraries that combine both approaches and create a hybrid style. The best example is MXNet, which allows imperative programs within symbolic programs as callbacks, or uses symbolic programs as a part of imperative programs. Another newer development is Eager Execution from Google's TensorFlow. Though, originally a Python library with a symbolic paradigm (a static computational graph of tensors), Eager Execution does not need a graph, and execution can happen immediately.

- Symbolic: Spark MLlib, MMLSpark, BigDL, CNTK, H2O.ai, Keras, Caffe2
- Imperative: scikit-learn, auto sklearn, TPOT, PyTorch
- Hybrid: MXNet, TensorFlow

Get O'Reilly's weekly data newsletter



2. Machine learning algorithms

Supervised learning, unsupervised learning, recommendation systems, and deep learning are the common classes of problems that we deal with in machine learning. Again, your use case will dictate which library to use. For example, if you are doing a lot of custom image processing, Caffe2 would be a good choice, all other factors being equal. It is an evolution of Caffe, whose original use case was CNN for image classification. CNTK would be a reasonable choice for language processing, as the CNTK framework was born out of the language services division of Microsoft. On the other hand, if most of the use cases are supervised and unsupervised learning, Spark MLlib, scikit-learn, H2O.ai, and MMLSpark are good alternatives, as they support an exhaustive collection of supervised and unsupervised algorithms. Spark MLlib, H2O.ai, and Mahout additionally support recommendations via collaborative filtering.

Many of the older libraries now fall short, with the rise of deep learning (DL). TensorFlow was one of the first libraries that made deep learning accessible to data scientists. Today, we have many others that are focusing on deep learning, including PyTorch, Keras, MXNet, Caffe2, CNTK and BigDL. There are other libraries that support DL algorithms,

but it is not a main function for them, such as MMLSpark (image and text learning) and H2O.ai (via the deepwater plugin).

- Supervised and unsupervised: Spark MLlib, scikit-learn, H2O.ai, MMLSpark, Mahout
- Deep learning: TensorFlow, PyTorch, Caffe2 (image), Keras, MXNet, CNTK, BigDL, MMLSpark (image and text), H2O.ai (via the deepwater plugin)
- Recommendation system: Spark MLlib, H2O.ai (via the sparkling-water plugin), Mahout

3. Hardware and performance

Compute performance is a key criteria in selecting the right library for your project. This is more predominant with libraries specializing in DL algorithms, as they tend be computationally intensive.

One of the biggest trends that has boosted DL development is advances in GPUs and being able to perform large matrix operations on GPUs. All DL libraries, such as TensorFlow, Keras, PyTorch, and Caffe2, support GPUs, but many general purpose libraries, like MMLSpark, H2O.ai, and Apache Mahout, support GPUs as well. CNTK and MXNet boast automatic multi-GPU and multi-server support, which allows for fast distributed training using parallelization across multiple GPUs without any need for configuration. TensorFlow, however, has gathered quite a bit of reputation as being slower than comparative DL platforms. As a compensation, TensorFlow is advertising big performance gains on their new custom AI chip, Tensor Processing Unit (TPU). The drawback being, TPU is non-commodity hardware and works only with TensorFlow, causing vendor lock-in.

Caffe2, MXNet, and TensorFlow also stand out for their mobile computation support—so if your use case requires running ML training on mobile, these libraries would be your best bet.

The takeaway on performance is that most libraries built on top of Spark are able to exploit the parallel cluster computing of Spark with cached intermediate data in memory, making machine learning algorithms that are inherently iterative in nature run fast. Apache Mahout is the exception, which only supported Hadoop MapReduce until recently and involves expensive disk I/Os, and hence was slower for iterative algorithms. Mahout now added Scala on Spark, H2O.ai, and Apache Flink support. BigDL is novel in its

approach of making DL possible on the Spark ecosystem with CPUs, a departure from traditional DL libraries, which all leverage GPU acceleration. They in turn use Intel's MKL and multi-threaded programming.

- CPU: Spark MLlib, scikit-learn, auto sklearn, TPOT, BigDL
- GPU: Keras, PyTorch, Caffe2, MMLSpark, H2O.ai, Mahout, CNTK, MXNet, TensorFlow
- Mobile Computing: MXNet, TensorFlow, Caffe2

4. Interpretability

ML software differs from traditional software in the sense that the behavior or outcome is not easily predictable. Unlike rule-based engines, such software constantly learns new rules. One of the biggest challenges we face at Salesforce Einstein is how to constantly build trust and confidence in machine learning applications. Why did it predict Lead X as having a higher likelihood of conversion to an opportunity while Lead Y has a lower likelihood? What are the patterns in the data set that are driving certain predictions? Can we convert such insights from the machine learning model into actions?

Other corollaries to this problem include visualizing computational graph execution metrics, observing data flows in order to optimize, and hand-craft models and/or debug model quality performance.

This is a relatively unripe area in ML, which only a few of the libraries make attempts to solve. H2O.ai launched Machine Learning Interpretability, which addresses some aspects of this problem. TensorFlow has a visualization layer called TensorBoard, which helps data scientists to understand, optimize, and debug massive deep neural networks. Keras also addresses this with their model visualization.

Though these are good steps in the right direction, this area needs more investment to make ML transparent and less of a black box in order to encourage wider adoption.

- Interpretability: TensorFlow (TensorBoard), H2O.ai (Machine Learning Interpretability), Keras (model visualization)

5. Automated machine learning

Arguably, one of the biggest area for innovation is automated machine learning. Real-life ML is not just about building models, but about building pipelines that include ETL, feature engineering, feature selection, model selection (including hyper-parameter tuning), model updates, and deployment.

Many of these workflows are common across applications and data sets, and tend to be repeated, meaning there is an opportunity to optimize and automate. Additionally, some of the workflows need significant intuition and tribal knowledge in data science and machine learning, such as feature engineering or tuning deep models. These make machine learning inaccessible to those who do not necessarily have a Ph.D. Automating many of the steps can accelerate data scientists' productivity and help build applications in hours rather than months.

Auto sklearn, TPOT, and H2O.ai are built on this premise, targeting supervised classification problems. Auto sklearn is automating model selection and hyper-parameter tuning using Bayesian optimization. TPOT is using genetic programming for their hyper-parameter tuning. Both TPOT and H2O.ai have also included several degrees of automation for feature engineering. MMLSpark has auto model selection and a certain degree of automated feature engineering for image and text features.

There is a large gap in the market for this category, both in the breadth (the different stages in the pipeline) and depth (intelligent approaches to automate a single stage) of offerings.

- Hyper-parameter tuning: auto sklearn (Bayesian optimization), TPOT (genetic programming)
- Limited auto feature engineering: TPOT, H2O.ai, MMLSpark

Other noteworthy considerations

Though models in ML need data sets to be trained on before they can be used, there are scenarios where one can get access to models for data sets that are global in nature. For example, a universal image data set like ImageNet is good enough for building a general-purpose image classification model, also known as a pre-trained model. Such models can be plugged in, meaning no data or training is needed. MMLSpark, CNTK, TensorFlow, PyTorch, Keras and BigDL all provide pre-trained models for general-purpose

classification tasks. One caveat here is that such models are useless for custom use cases. For instance, a general-purpose image classification model will perform poorly if it needs to classify the type of crops from aerial images of crop fields, but it would work well classifying cats versus dogs. This is because, though ImageNet has crop images, there is insufficient training data of specific types of crops or crops with different diseases that, for instance, a fertilizer company might care about.

CNTK comes with some additional handy features like automatic randomization for data sets and real-time training. Though MMLSpark is a Scala library, it supports auto-generation of interfaces in other languages, namely Python and R.

- Pre-trained models: MMLSpark, CNTK, TensorFlow, PyTorch, Keras, BigDL
- Real-time training: CNTK
- Multi-language support: MMLSpark supports auto generation of interfaces in Python/R from Scala

Decision time

There are myriad options for ML libraries to choose from when you are building ML into a product, and while there may not be one perfect option, it helps to consider the above factors to ensure you're picking the best solution for your specific needs. For enterprise companies that have thousands of business customers, there are a host of other challenges that the market does not yet address. Label leakage, also known as data leakage, has been the Achilles' heel of ML libraries—this occurs when, because of unknown business processes, the data set available for model training has fields that are proxy for the actual label. Sanity checking the data for such leaks and dropping them from the data is key to well-performing models.

Multitenancy is another sticking point—how can we share common pieces of machine learning platforms and resources to serve multiple tenants, each of which has its own unique data sets and leads to completely different models being trained? This problem lends itself to scale of a different sort. As the industry continues to face challenges like this head on, a complete and exhaustive auto ML solution that has yet to be developed will likely prove to be the key to success.

Article image: Main reading room at the Library of Congress (source: Library of Congress on Flickr).